# Agenda

## Advanced Components

- Component Inheritance
- Nested Components
- Content Projection
- ViewChild
- Lifecycle Hooks
- Change Detection

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

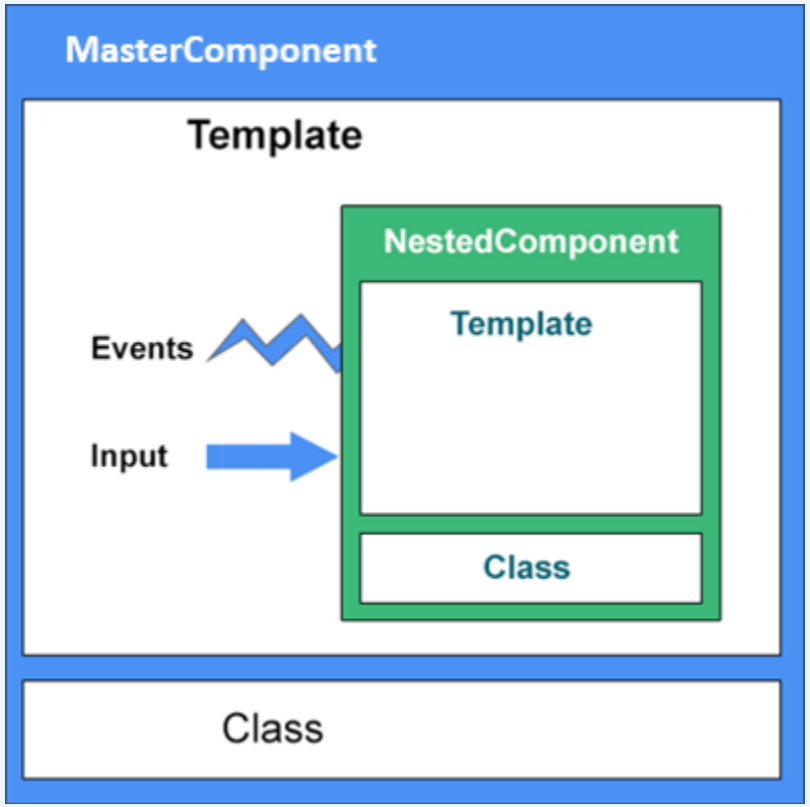
# Components Inheritance

- **Metadata**: Metadata (@Input, @Output) defined in child class will override base class metadata, else the base class metadata will be used. Metadata (@Component, @Directive, @NgModule etc.) are not inherited by child.
- **Constructor**: The base class constructor will be used if the derived class doesn't have one. This means that all the services injected in parent constructor will be inherited to child component as well.
- **Lifecycle hooks**: Parent component lifecycle hooks (ngOnInit, ngOnChanges etc.) will be called when they are not defined in derived class.

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

# Nested Components

- Nested components can be created to create master/detail view
- @Input decorator is used to pass data from master to nested component
- @Output decorator is used to pass data from nested to master component with the help of events

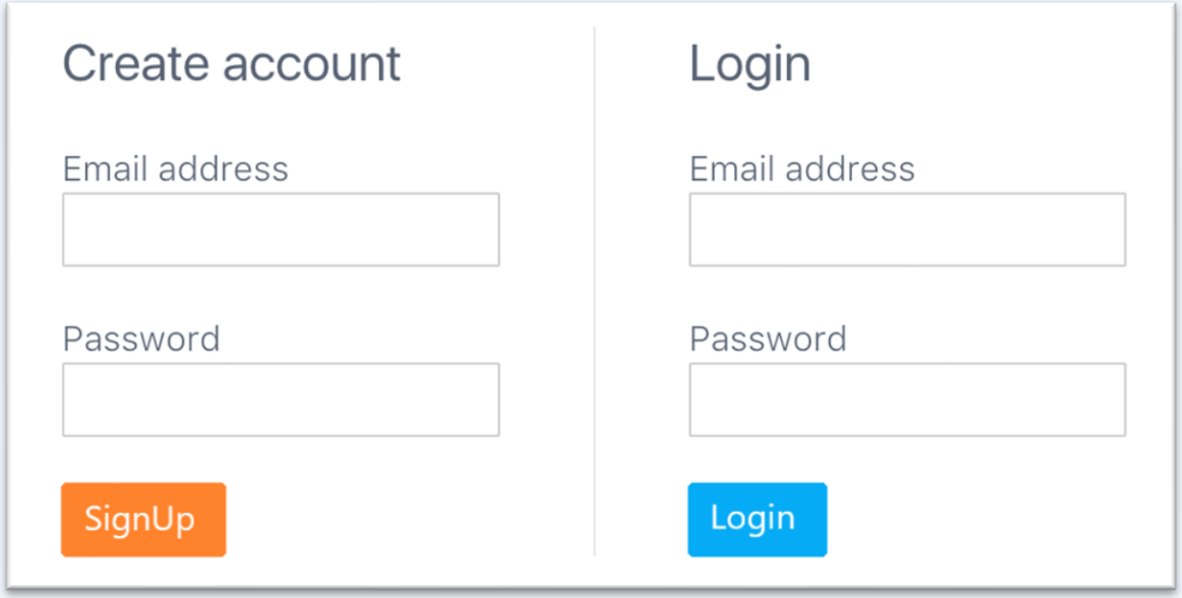



Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

# Content Projection

Enables developers to build reusable components and make applications more scalable and flexible.

Cards, tabs, navbars, modals and dialogs are some commonly used UI elements which can accept generic markups or content.




Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

# <ng-content> Selectors

- Html Tag Selector
- CSS Class Selector
- Id Selector
- Attribute Selector

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

# ViewChild

- Useful to access a directive, child component, or a DOM element inside a parent component class.
- The ViewChild decorator returns the first element that matches a given directive, component, or template reference selector.

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-child',
  templateUrl: './child.component.html'
})
export class ChildComponent implements OnInit {
  constructor() { }
  ngOnInit(): void {}
  Hello() {
    return 'I am a child component!';
  }
}
```

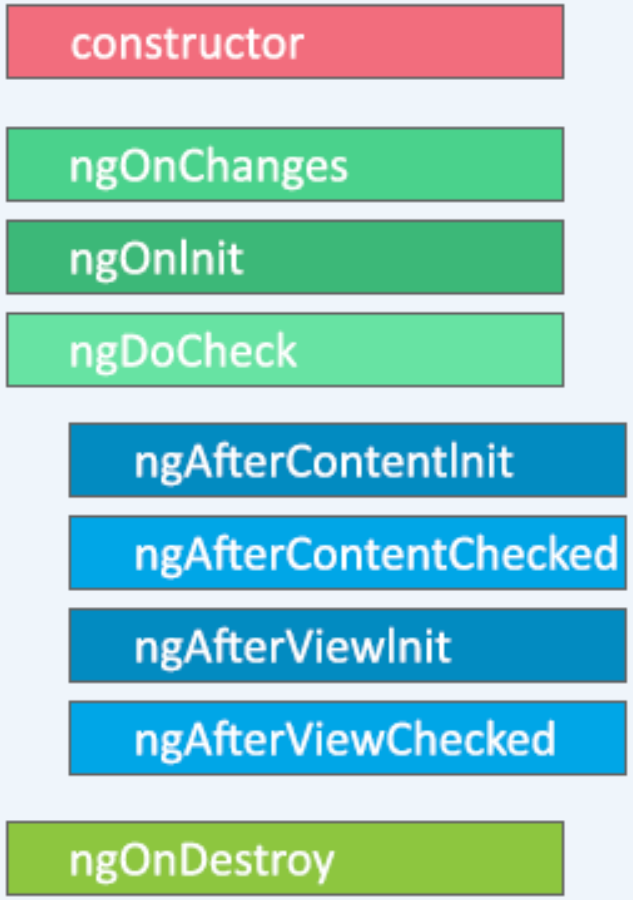
```
import { Component,ViewChild,AfterViewInit } from '@angular/core';
import { ChildComponent } from './child/child.component';

@Component({
  selector: 'app-root',
  template: `<app-child></app-child>`
})
export class AppComponent implements AfterViewInit {
  @ViewChild(ChildComponent) child: ChildComponent;
  ngAfterViewInit() {
    console.log(this.child.Hello());
  }
}
```

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

# Component Lifecycle Hooks

- Component lifecycle is managed by Angular itself
- Angular manages creation, rendering, data-bound properties etc.
- Angular offers given lifecycle hooks that allow you to execute custom logic
- The hooks are executed in the order as shown in fig.




Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

# Hooks for the Component

- **constructor** - Invoked when Angular creates a component or directive by calling *new* on the class.
- **ngOnChanges** - Invoked every time there is a change in one of the input properties of the component.
- **ngOnInit** - Invoked when given component is initialized. This hook is only called once after the first ngOnChanges.
- **ngDoCheck** - Invoked when the change detector of the given component is invoked. It allows us to implement our own change detection algorithm for the given component.
- **ngOnDestroy** - Invoked just before destroying the component. Useful to unsubscribe observables & detach event handlers to avoid memory leaks.

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

# Hooks for the Component’s Child

- **ngAfterContentInit** - Invoked after the child components (Projected using <ng-content>) of a component is initialized.
- **ngAfterContentChecked** - Invoked each time when child components (Projected using <ng-content>) of a component is checked by the Angular change detection mechanism.
- **ngAfterViewInit** - Invoked after the component’s child components (Access by @ViewChild) are initialized.
- **ngAfterViewChecked** - Invoked each time when child components (Access by @ViewChild) of a component is checked by the Angular change detection mechanism.
- **Note -** These hooks are only called for components and not for directives.

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

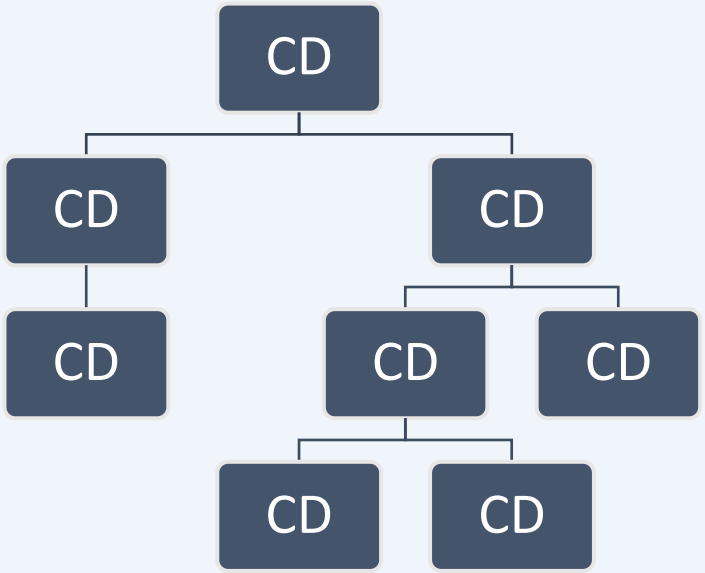
# Zone.js

- Provides a mechanism, called zones, for encapsulating and intercepting async activities in a browser (setTimeout, promises)
- Angular *NgZone* service is based on Zone.js, which is used to automatically trigger change detection as a result of async operations/activities

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

# Change Detection (CD)

- A technique to detect changes in angular app component and then automatically re-render the view to reflect that change
- Every component has its own change detector which is generated at run time to improve performance
- CD is unidirectional and makes a single pass in component tree from top to bottom, when one of the components change




Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

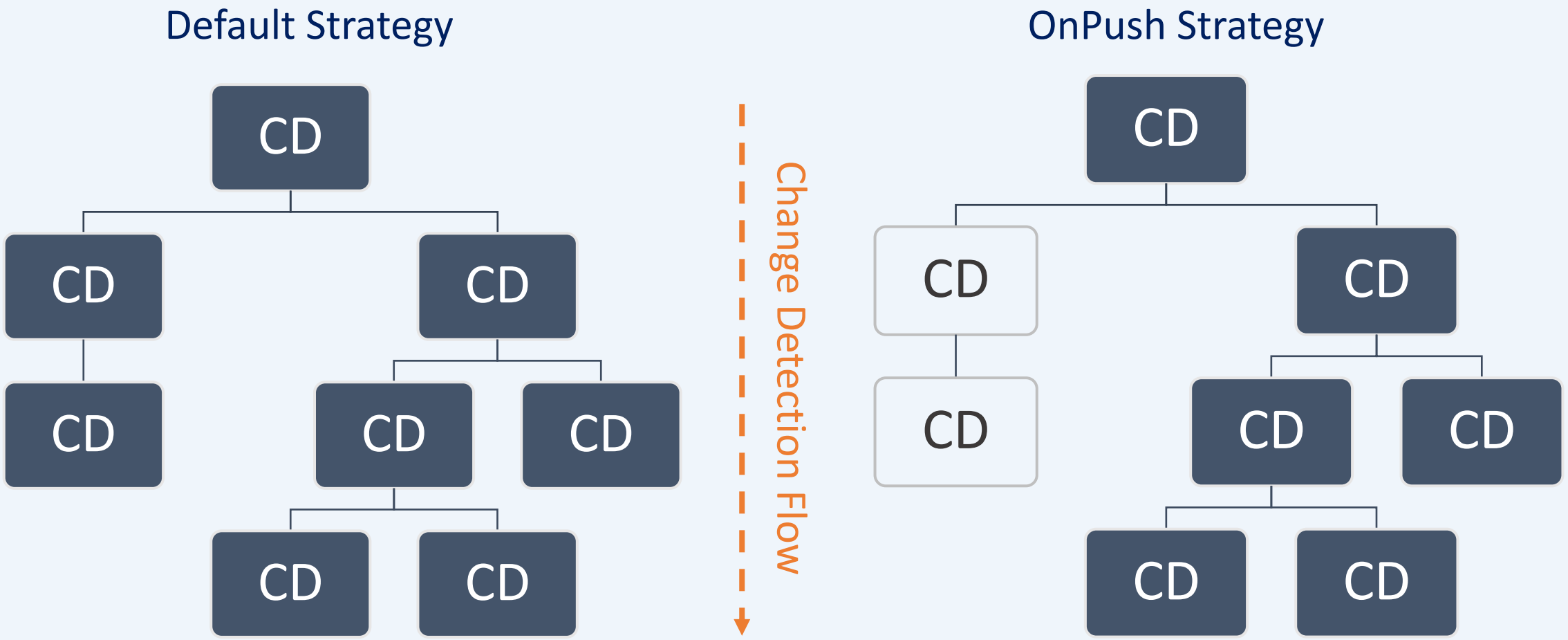
ScholarHat

# Change Detection Strategies

- Default
  - The change detector's mode is set to *CheckAlways*
  - Update the component every time when data change
- OnPush
  - The change detector's mode is set to *CheckOnce*
  - Update the component only when its inputs change or the component requested to be updated

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

# Change Detection Strategies Contd.




Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

ScholarHat